

ASSIGNMENT - 3

CSC-303 – Computer Networks

Group 4 -

24114005	Aditya Bhadoria	Q1, Q2
24114006	Aditya Ranjan	Q3, Q4
24114028	Daddi Om	Q1, Q3
24114032	Deokar Parth	Q2, Q4
24114050	Kothawade Manthan	Q1, Q4
24114088	Satyam Sharma	Q2, Q3

Que 1: DNS Caching and Network Performance

Mechanism of DNS Caching: The Domain Name System (DNS) acts like the phonebook of the internet, translating human-friendly domain names (like www.example.com) into machine-readable IP addresses. Because looking up this "phone number" every single time you visit a webpage would be slow, systems use a mechanism called DNS caching. When you visit a site, your web browser, your computer's operating system, or your Internet Service Provider (ISP) temporarily saves (caches) the IP address. The next time you try to load that page, the system retrieves the IP address directly from its local memory instead of asking the global DNS servers again.

Benefits in Large-Scale Systems:

- **Reduced Latency:** By skipping the entire DNS lookup process, web pages begin loading almost instantly for the user.
- **Network Efficiency:** Caching drastically reduces the volume of query traffic traveling across the internet. In large-scale systems, like Content Delivery Networks (CDNs) or major ISPs, caching prevents root DNS servers from being overwhelmed by millions of redundant requests during traffic spikes.

Drawbacks and Challenges:

- **Stale Records:** If a website changes its server and gets a new IP address, the cached data becomes outdated (stale). Users might temporarily be unable to reach the site until the cache expires and refreshes.
- **Security Risks:** Caches can be targeted by "DNS spoofing" or "cache poisoning." Hackers can secretly replace a legitimate IP address in the cache with a malicious one, silently redirecting users to fake websites.

Que 2: HTTP vs. FTP for File Retrieval

Core Differences and Design: Both the Hypertext Transfer Protocol (HTTP) and the File Transfer Protocol (FTP) are used to move data over the internet, but they were built for very different purposes. HTTP was designed as the foundation of the World Wide Web, meant to quickly fetch and display linked text and multimedia. FTP is an older protocol designed explicitly for moving, copying, and managing files between a client and a server.

Scenarios and Use Cases:

- **HTTP (The Best for General Use):** HTTP is preferable for everyday, casual file retrieval. If a user needs to download a PDF, an image, or a software installer from a web page, HTTP is ideal. It requires no special software (any web browser can handle it), and it seamlessly integrates with modern security standards like HTTPS.
- **FTP (The Best for Heavy Lifting):** FTP is preferable for bulk file management. If a web developer needs to upload thousands of files to a server, or if a user is downloading massive datasets that take hours, FTP is the better tool. It naturally supports pausing, resuming, and managing directory structures.

Trade-offs (Security and Speed): While FTP is excellent for continuous, large data streams, it is inherently insecure in its original form, sending passwords and data in plain text (though modern alternatives like SFTP solve this). HTTP is incredibly secure and user-friendly but carries a bit more overhead data, making it less efficient for managing complex server files.

Que 3: Cookies in Web Applications and Privacy

The Role of Cookies: By default, the HTTP protocol is "stateless," meaning it has no memory. Every time you click a new link on a website, the server treats you like a brand-new visitor. To solve this, developers use cookies - small text files saved by your web browser. Cookies provide the necessary "memory" for modern web applications, allowing sites to keep you logged in, remember what is in your shopping cart, and save your language preferences.

Privacy Concerns and GDPR Influence:

- **The Tracking Problem:** While "first-party" cookies (set by the site you are visiting) are usually helpful, "third-party" cookies (set by advertisers) can track your browsing behavior across multiple different websites, building a detailed profile of your habits without your explicit knowledge.
- **Regulatory Impact (GDPR):** The General Data Protection Regulation (GDPR) fundamentally changed how cookies operate by mandating user consent. Websites can no longer legally place non-essential tracking or marketing cookies on a user's device by default. Users must now explicitly "opt-in" through cookie consent banners.

Best Practices for Developers: To balance functionality and privacy, developers should only collect the absolute minimum data required for the site to function. Furthermore, developers must secure their cookies by adding tags like Secure (ensuring the cookie is only sent over encrypted HTTPS connections) and HttpOnly (preventing malicious scripts from stealing the cookie data).

Que 4: SMTP Vulnerabilities and Email Security

The Role of SMTP: The Simple Mail Transfer Protocol (SMTP) is the standard protocol used for sending emails across the internet. It acts as the digital mail carrier that moves your message from your outbox to the recipient's mail server.

Inherent Vulnerabilities: SMTP was designed in the early days of the internet when the primary goal was simply getting messages to arrive reliably. As a result, it has no built-in security. By default, SMTP sends emails in plain text, making them easy to intercept and read. More critically, original SMTP lacks sender authentication; it is incredibly easy for a malicious user to "spoof" an email, making it look exactly like it came from your bank or your boss.

Securing Communications: To fix these flaws, modern email systems layer additional security protocols on top of SMTP:

- **Encryption:** SSL/TLS is used to encrypt the connection between mail servers, ensuring no one can read the emails while they are in transit.

- **Authentication (SPF, DKIM, DMARC):** These are advanced DNS records that verify the sender's identity. They act like a digital wax seal, proving that the email actually originated from the domain it claims to be from and that the message wasn't altered along the way.

Balancing Security and Ease of Use: Implementing strict email security requires a delicate balance. If a server administrator configures their security rules (like DMARC) too aggressively, legitimate emails, such as automated password resets or newsletters, might be incorrectly flagged as spoofed and blocked. The challenge is setting the filters high enough to stop phishing attacks, but forgiving enough to ensure users do not miss critical communications.